
DSC 140A - Homework 01

Homework 01 Instructions. You may collaborate with whomever, including AI. Each student must submit their own homework even if they worked with others. We will not be grading for correctness, but we will be grading for completeness.

Problem 1.

In this problem, we'll show that the gradient of $f(\mathbf{x}) = \mathbf{x}^T \mathbf{A} \mathbf{x}$ is $2\mathbf{A}\mathbf{x}$, where $\mathbf{A}$ is a symmetric $n \times n$ matrix and $\mathbf{x} \in \mathbb{R}^n$. This is a useful result, but it's also a good exercise for reviewing topics in matrix-vector algebra and multivariate calculus.

- a) To compute the gradient, we need to compute $\partial f / \partial x_1$, $\partial f / \partial x_2$, and so on. To do this, we'll start by expanding $\mathbf{x}^T \mathbf{A} \mathbf{x}$ until we see the coordinates of $\mathbf{x}$.

Let the entries of $\mathbf{A}$ be

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}.$$

Show that $f(\mathbf{x}) = \mathbf{x}^T \mathbf{A} \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j$.

- b) Show that:

$$\frac{\partial}{\partial x_k} \left(\sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j \right) = 2 \sum_{j=1}^n a_{kj} x_j$$

In other words, show that $\partial f / \partial x_1$ is $2 \sum_{j=1}^n a_{1j} x_j$,
 $\partial f / \partial x_2$ is $2 \sum_{j=1}^n a_{2j} x_j$, and so on.

- c) The gradient vector is the vector of partial derivatives:

$$(\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n)^T$$

We have found so far that the gradient is:

$$\begin{pmatrix} 2 \sum_{j=1}^n a_{1j} x_j \\ 2 \sum_{j=1}^n a_{2j} x_j \\ \vdots \\ 2 \sum_{j=1}^n a_{nj} x_j \end{pmatrix}$$

Show that this is equal to $2\mathbf{A}\mathbf{x}$.

Hint: You can work backwards, expanding $2\mathbf{A}\mathbf{x}$ and showing that it equals the gradient.

Problem 2.

Let $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$ be a set of n training examples, where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$. Recall that the mean squared error of a linear predictor is defined to be

$$R(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\text{Aug}(\mathbf{x}_i)^T \mathbf{w} - y_i)^2.$$

In lecture, we saw two equivalent expressions for the gradient of $R(\mathbf{w})$:

$$\frac{dR}{d\mathbf{w}} = \frac{2}{n} \sum_{i=1}^n (\text{Aug}(\mathbf{x}_i)^T \mathbf{w} - y_i) \text{Aug}(\mathbf{x}_i),$$

and, in matrix-vector form:

$$\frac{dR}{d\mathbf{w}} = \frac{2}{n} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}),$$

where $\mathbf{X}$ is the $n \times (d+1)$ *design matrix* whose i th row is $\text{Aug}(\mathbf{x}_i)$, and $\mathbf{y}$ is the $n \times 1$ vector whose i th entry is y_i .

Show that these two expressions are equivalent.

Problem 3.

In lecture, we derived the least squares solutions for linear prediction rules $H(x) = w_1x + w_0$. They were:

$$w_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$w_0 = \bar{y} - w_1 \bar{x}$$

Where $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ and $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$.

You may see these solutions written in various equivalent forms. In this problem, we'll derive another form that you may find useful in solving other problems.

- a) Show that $\sum_{i=1}^n (x_i - \bar{x}) = 0$.
- b) Use the result of the previous part to show that

$$w_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})y_i}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

is equivalent to the formula for w_1 that was given in lecture.

Problem 4.

In practice, the performance of nearest neighbor predictors often decreases with the number of features. This is often attributed to the so-called “curse of dimensionality”. One informal statement of the curse goes: “in high dimensions, almost all points in a randomly-drawn set of points are essentially equidistant from the origin.”

In this problem, you'll demonstrate this empirically. For each value in a sequence of increasing d (for example, $d = 2, 4, 8, 16, \dots$), generate a data set of 1,000 points in $\mathbb{R}^d$, where each coordinate of each point is drawn

from the uniform distribution on the interval $[-1, 1]$. That is, for any given d , your data set should consist of 1,000 draws from the uniform distribution on the d -dimensional hypercube $[-1, 1]^d$.

Use your datasets to generate the following plots. Use python and paste your code in your solution. Lastly, it's up to you to decide how large your sequence of d should be – just make sure it's large enough to see the trend.

- a) Let $\Delta_0(d)$ be the distance of the **closest** point to the origin in your data set of dimensionality d . Plot $\Delta_0(d)$ as a function of d .
- b) Let $\Delta_1(d)$ be the distance from the origin to the **furthest** point in your data set of dimensionality d . Plot the ratio

$$\frac{\Delta_1(d)}{\Delta_0(d)}$$

for your sequence of increasing d .

Problem 5.

In this problem, you'll implement a k -nearest neighbor classifier and use it to predict whether an image of a handwritten digit (like the one shown below) is either a 3 or a 7.



Along the way, you'll see some of the common technical issues that might come with using a machine learning algorithm and how to fix them.

The file linked below is a subset of the MNIST dataset, which contains images of handwritten digits. It is provided in `npz` format, which is a compressed file format that can store multiple arrays; it can be read using the `numpy` library's `np.load` function.

<https://f000.backblazeb2.com/file/jeldridge-data/014-mnist37/mnist.npz>

When loaded it behaves like a Python dictionary with the following keys:

- **train**: A 784 by 12,396 array containing the training images. Each column represents one training image that has been “flattened” into a 784-dimensional vector (784 dimensional because the original images are 28 by 28 pixels, and 28 times 28 is 784).
- **train_labels**: An array of 12396 entries containing the labels for the training images. Each entry is either a 3 or a 7.
- **test**: A 784 by 2038 array containing the test images. Each column represents one test image that has been flattened into a 784-dimensional vector.
- **test_labels**: An array of 2038 entries containing the labels for the test images. Each entry is either a 3 or a 7.

¹If you'd like to plot the handwritten digits (for fun), you can use `plt.imshow(x.reshape(28, 28))` from the `matplotlib` library, where `x` is a 784-dimensional vector.

In lecture, code was given for a function `knn_predict` that implemented a k -nearest neighbor classifier. You're encouraged to use that code for this problem, but if you simply plug in the training and test data, you'll find that the classifier doesn't work as expected. There isn't a bug in the code; rather, the issue is that the input is not in a form that the code expects. In that sense, this problem is meant to be practice for how to manage these small technical issues (since you will likely encounter them at some point, either in this class or in your career).

Some errors you might get are:

- “**ValueError**: operands could not be broadcast together with shapes ...”: this error occurs when you try to add or subtract two arrays of different shapes. Check: does the `knn_train` function expect the training data to be in a $n \times d$ matrix, or a $d \times n$ matrix? What are you providing it with?
- “**TypeError**: Cannot cast array data from dtype('float64') to dtype('int64')...”: This error is probably being raised by `np.bincount`. It wants an array of integers. Is that what you're giving it? You might want to use the `.astype()` method to convert an array to a different data type.

After fixing these errors, your code should run – but we're not done just yet. Compute the accuracy of your classifier on the first 100 test examples using $k = 1$ neighbors. You will likely see something like 81% accuracy. This might sound good, but it's actually much less than it should be.

The issue is that the training images are all encoded as 8-bit “unsigned integers” (or `uint8`, in `numpy`'s language), and an 8-bit unsigned integer can only store values from 0 to 255. If you subtract two 8 bit integers (as we do when we compute the distance between two images), the result might be a negative number *mathematically*, but the computer will not be able to store the result as a negative number in an 8-bit unsigned integer. Instead, it will “wrap around” and store the result as a positive number. This means that the distance between two images is not what you'd expect. For example, if you compute:

```
>>> x = np.array([1, 2, 3], dtype=np.uint8)
>>> y = np.array([4, 5, 6], dtype=np.uint8)
>>> x - y
```

you expect to get `array([-3, -3, -3])`, but you actually get `array([253, 253, 253], dtype=uint8)`.

To fix this, you should convert the training and test images to a data type that can represent negative numbers as large as needed. `np.int16` should be enough, since it can represent numbers from $-32,768$ to $32,767$.

After fixing all of these, you should have a classifier that gets very close to 100% accuracy on the first 100 test examples. *Hint*: you should not need to change the code of `knn_predict` at all; you just need to change its inputs.

What you should turn in for this problem:

- the final accuracy of the $k = 1$ nearest neighbor classifier on the first 100 test examples after fixing the data type issue.
- The predictions your classifier makes on the 0th, 500th, and 1000th test examples. E.g.,: 7, 7, 7 if the classifier predicts 7 for all three examples (which it shouldn't!)
- A short list of the changes you had to make to get these results.
- Your code, either as a screenshot or pasted into the document.